Data Serialization and Deserialization in Distributed Systems

Lab report: JSON, Java Serialization and Google Protocol Buffers

Name: Rhiana Sakkour

1. Setup and scenario

The lab is built around a small online store. The shop wants to analyse what its customers bought before the holiday season, so a customer purchase profile has to travel between the nodes of a distributed system. The profile holds the customer name and city, a list of product names, a list of prices and a flag saying whether the customer qualifies for holiday offers. It also carries a temporary session token that is only used for the discount calculation and must never leave the machine. The three experiments move this kind of object with JSON, with standard Java serialization and with Protocol Buffers, and then compare the results.

I ran everything on Windows 11 from PowerShell. The programs run on OpenJDK 11.0.12, which is the `java` on my PATH, while Maven 3.10.0 runs on JDK 17.0.21. The project is compiled with `release 11`, so the two agree (Figure 1). JSON is handled with Jackson 2.17.2 and the Protocol Buffers part uses protobuf-java 3.25.5.

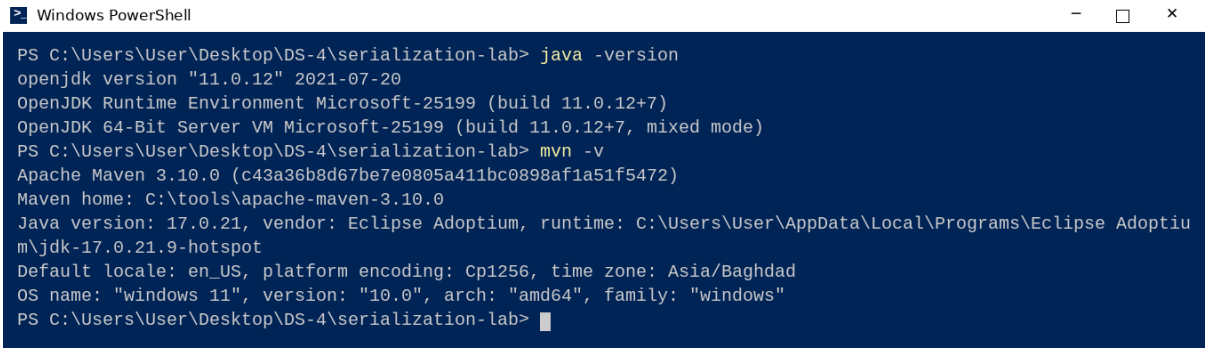


Figure 1. Java and Maven versions on the machine used for the lab.

The lab suggests calling the compiler by hand with `protoc --java_out=./src/main/java customer_profile.proto`. Instead I let the Maven protobuf plugin run protoc, which it downloads by itself (`protoc --version` reports libprotoc 25.5, the release that goes with protobuf-java 3.25.5), so nothing has to be installed and the build can be repeated on any machine. The generated class `CustomerProfileProto.java` ends up in `target/generated-sources/protobuf/java/com/distributed/lab/`. It is the same stub the manual command gives, only in a different folder (I also ran the manual command, see Task 3.1).

Everything is one Maven project. The script `run-all.bat` runs each experiment and saves what it prints in a text file inside `results`. Figure 2 shows the script finishing and the files it created. All the output in this report was taken from those files, from the second complete run, which is the one I used for the numbers below.

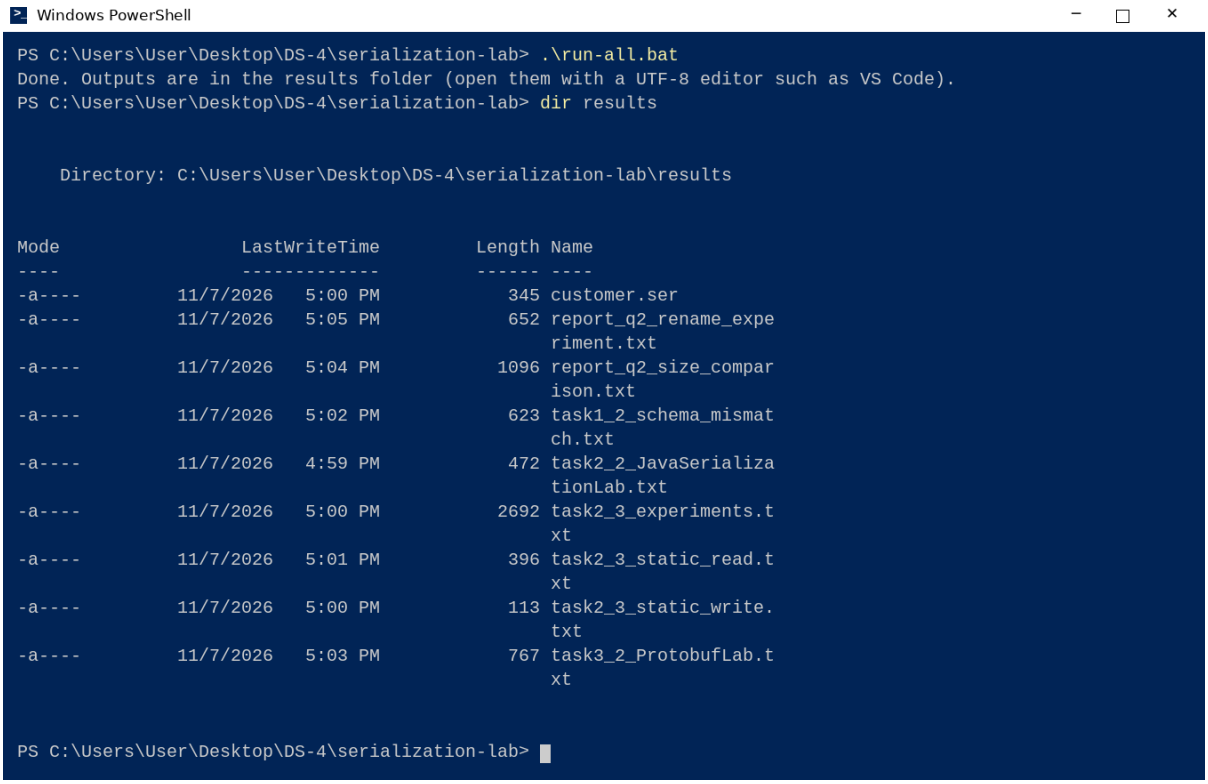


Figure 2. Running run-all.bat and the files it produced in the results folder.

2. Experiment 1: JSON and the missing schema

Task 1.1: building a JSON object and testing it

I created `customer_request.json` exactly as in the lab. It mixes a string, an array of objects (`pastOrders`), an array of strings, an array of integers and a boolean.

```
{
    "customerName": "Ahmad Saleh",
    "city": "Damascus",
    "pastOrders": [
        { "orderId": 101, "storeBranch": "Main" },
        { "orderId": 205, "storeBranch": "West" }
    ],
    "purchasedItems": ["Laptop", "Wireless Mouse", "Mechanical Keyboard"],
    "itemPrices": [1200, 25, 85],
    "isHolidayDealEligible": true
}
```

For the curl command of the lab to work, something has to be listening on port 8080, so I wrote a small server called `AnalyzeServer` on top of the HttpServer class that ships with the JDK, the same kind of server as in the previous lab. It accepts a POST on `/api/analyze`, checks the type of every field, adds up the prices and answers with a short JSON object containing the total, the number of items and a suggested promotion. The promotion rule is my own because the lab does not define one: 15 percent if the customer is eligible and the total is at least 1000, 10 percent if eligible with a lower total, and none otherwise. I started the server in one PowerShell window and sent the request from a second one. I used `curl.exe` because plain `curl` in Windows PowerShell is only an alias for Invoke-WebRequest.

```
curl.exe -X POST http://localhost:8080/api/analyze -H "Content-Type: application/json" -d
"@customer_request.json"
```

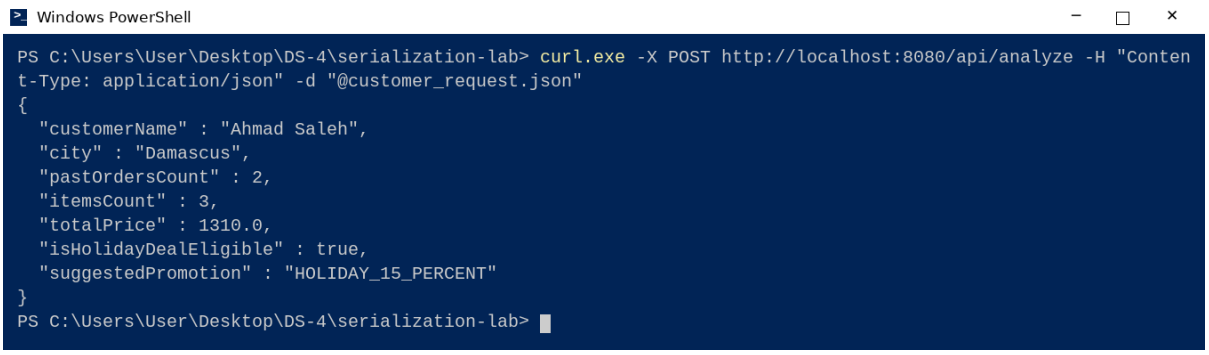


Figure 3. Response of AnalyzeServer to the curl request (second PowerShell window).

The server window only printed that it was listening on port 8080 with its two endpoints. The answer in Figure 3 is what I expected from the file: the total is 1310 (1200 + 25 + 85), there are 3 items and 2 past orders, and since the customer is eligible and the total is above 1000 the suggested promotion is the 15 percent one.

What stands out is how little is needed. The request is plain text that I can open in Notepad and edit by hand, and curl does not need to know anything about the customer class. The server does not need the sender's code either. Any tool that can send text over HTTP can take part, which is the main reason JSON is so common between browsers, scripts and servers.

I also wrote the same call in Java, `JsonHttpClientDemo`, with `HttpClient.sendAsync` and a `CompletableFuture` like in the previous lab. It posts the customer file and then the two price files from Task 1.2 one after the other (Figure 4).

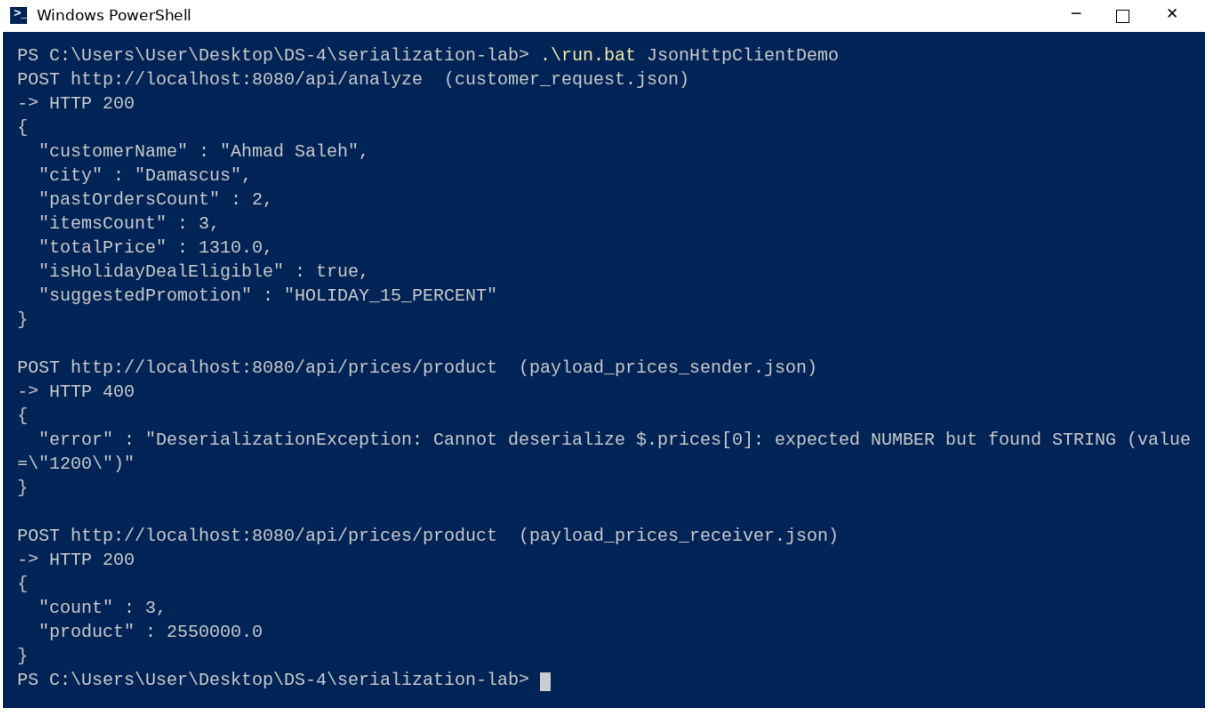


Figure 4. Output of JsonHttpClientDemo against the running server.

The type checks of the server are written by hand, so I also tried its error paths. The first bad body contains only a `customerName`, the second has the price as a string, `"10"`. The first comes back as

HTTP 400 with a list of four problems, the second as HTTP 400 with `itemPrices[0] must be a number`, and a plain GET on the endpoint gives HTTP 405 (Figure 5). It is a small example of the problem in the next task: all of these type rules live in my code and not in a document that both sides share.

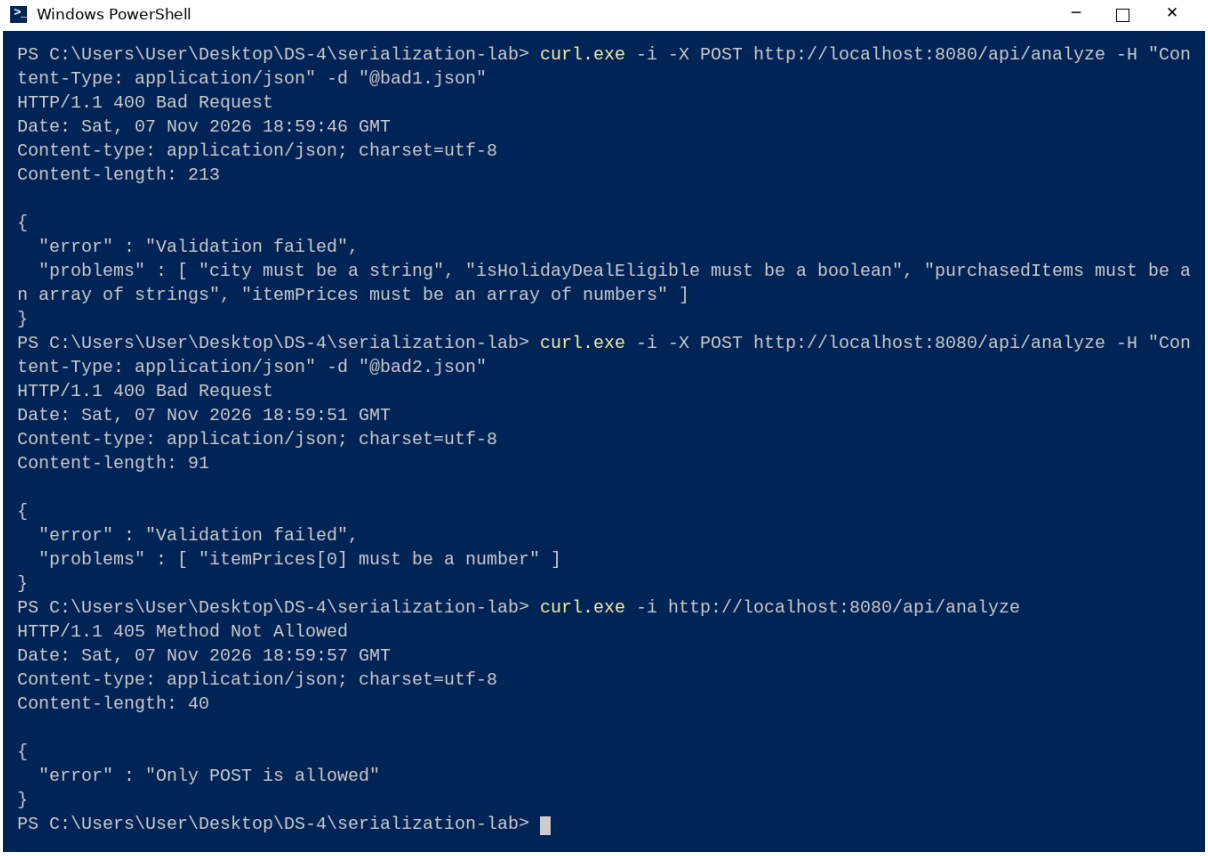


Figure 5. Error paths of /api/analyze: two invalid bodies (HTTP 400) and a GET request (HTTP 405).

Task 1.2: simulating the missing single source of truth

To reproduce the problem I wrote a receiver, `StrictPriceReceiver`, that expects `{"prices": [1200.00, 25.00, 85.00]}` and multiplies the numbers. It checks every element and throws a `DeserializationException` if one of them is not a JSON number. A second program, `SchemaMismatchDemo`, plays both sides: first it sends the receiver what the other service sends, the prices as strings `{"prices": ["1200", "25", "85"]}`, and then it sends the format the receiver expects. The result is in Figure 6.

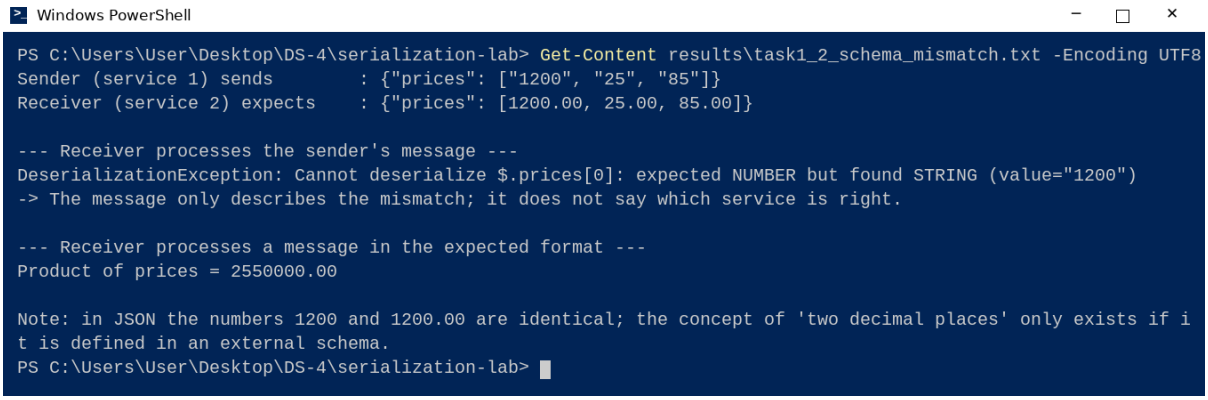


Figure 6. Output of SchemaMismatchDemo.

The string version is rejected at the very first element, and the message says a STRING was found where a NUMBER was expected. The numeric version goes through and gives 2550000.00, which is 1200 x 25 x 85.

The server uses the same receiver class on `/api/prices/product`, so I also posted the two payloads to it with curl. The string version comes back as HTTP 400 with the same error text, and the numeric one as HTTP 200 with a count of 3 and a product of 2550000.0 (Figure 7).

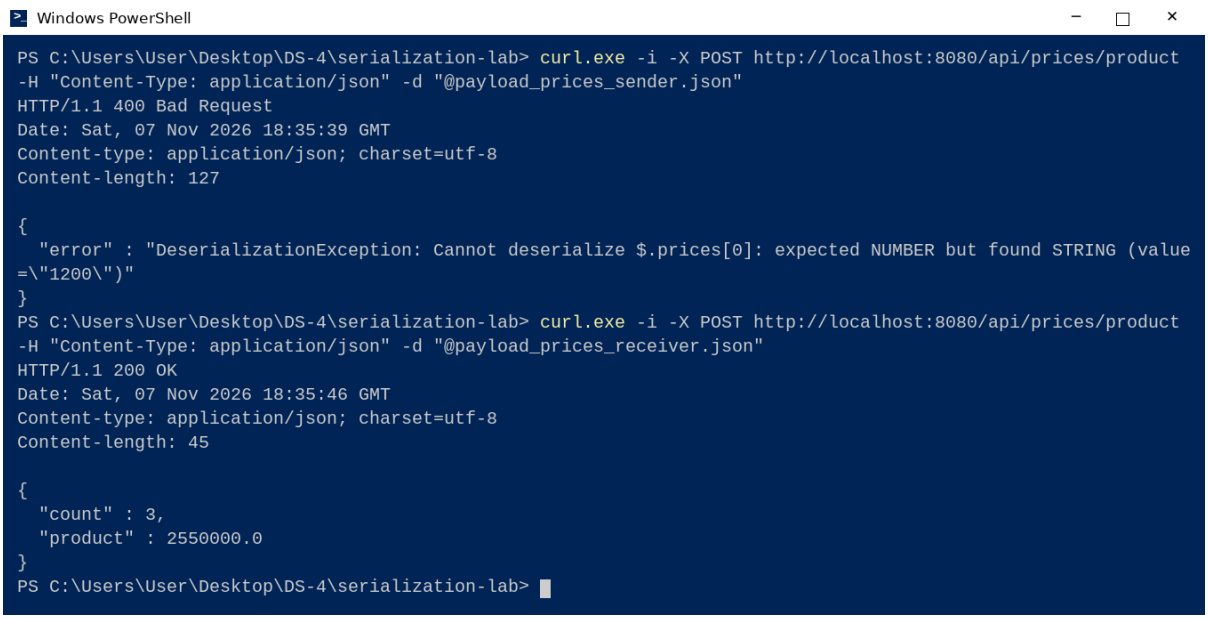


Figure 7. The two price payloads posted to /api/prices/product: HTTP 400 for the strings, HTTP 200 for the numbers.

Report question 1

Why can the on-call engineer not tell which service is correct from a DeserializationException, and how does the missing schema affect large distributed systems?

The exception only says that what arrived is not what the receiver wanted. That is a statement about a difference between two services, and the message would be exactly the same if the sender is wrong, if the receiver is wrong, or if both follow two different versions of the documentation. Nothing in the data settles it. Both messages are perfectly valid JSON. JSON tells you what shape a value has (string, number, array) but not what a field called `prices` is supposed to contain, so the agreement lives in the heads of two teams or on a wiki page that may be out of date. Even the lab's

"two decimal places" cannot be written in JSON, because 1200 and 1200.00 are the same number to any parser. That rule has to be stated somewhere else.

In a large system the problem multiplies. Every service depends on assumptions about its neighbours that nobody wrote down, and a small change on one side breaks callers its author may not even know about. Nothing is checked when the code is built, so the failure shows up in production, sometimes hours after a deployment. It can also be silent: some parsers quietly turn "1200" into 1200, and the wrong data looks fine until a total is off. With many services the number of these unwritten agreements grows roughly with the square of the number of services, and one mismatch can spread to everything that depends on the failing service. Rolling upgrades become risky because nobody can check automatically that a new version is compatible with the old one, and nothing can generate clients, contract tests or documentation from a single source.

The fix is to write the contract down once and check it automatically. That can be a JSON Schema or OpenAPI description, or a format like Protocol Buffers or Avro where the schema is part of the tool chain, together with compatibility checks in the build and a registry that holds the current version. Then the engineer on call has a document to compare both services against.

3. Experiment 2: standard Java serialization

Task 2.1: the serializable class

I wrote `CustomerProfile` as given in the lab. It implements `Serializable`, has `serialVersionUID = 1L`, a static field `systemVersion` ("v2.4"), a transient field `tempDiscountToken` and four normal fields: the name, the list of items, the array of prices and the eligibility flag. For the second experiment of Task 2.3 I also added a field `Address address`. It stays null in every normal run, so the other tasks behave as in the lab. The full listing is in the appendix.

One remark about the comment in the lab code, which says the empty constructor is needed to rebuild the object. For a class that implements `Serializable`, Java does not call its constructor when it restores an object. The only constructor that has to be accessible is the no-argument constructor of the first superclass that is not serializable, which here is `Object`. Keeping the constructor does no harm, it just is not what makes the restore work.

Task 2.2: serialization and deserialization

`JavaSerializationLab` writes the object into a byte array, changes the static field and then restores the object from the bytes. The output is in Figure 8. The payload is 345 bytes. The token `SECRET-TOKEN-999` is there before serialization and `null` afterwards, and `systemVersion` shows the new value I assigned in between.

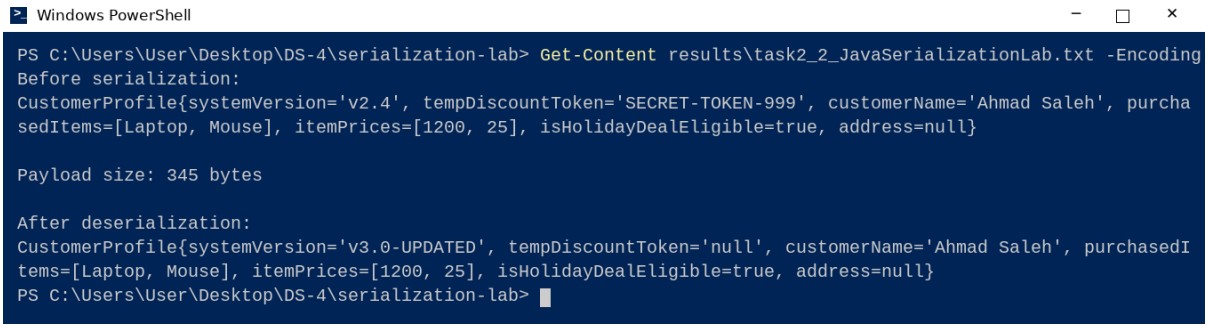


Figure 8. Output of JavaSerializationLab.

Task 2.3: exploration experiments

The three experiments are in `Task23Experiments`. Figure 9 has the output of the first two.

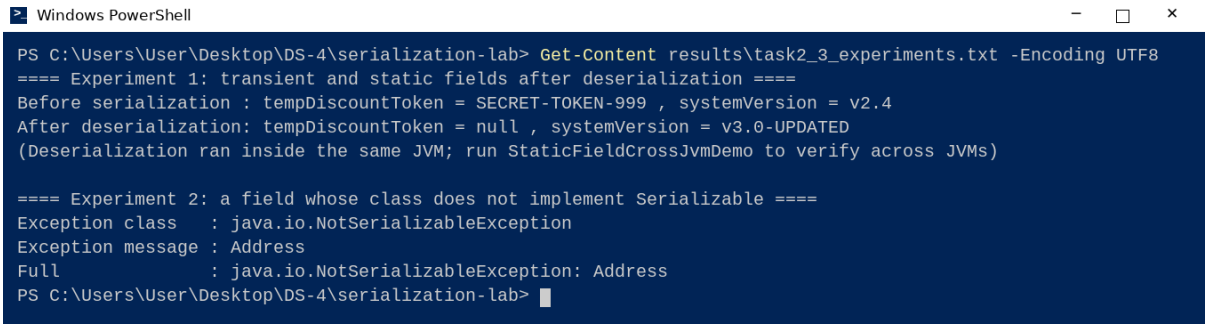


Figure 9. Experiments 1 and 2 of Task 2.3 (transient/static fields and the non-serializable Address).

Experiment 1: tempDiscountToken and systemVersion after restoring the object

After the restore `tempDiscountToken` is `null` and `systemVersion` is `v3.0-UPDATED`. The token is declared `transient`, so the stream never contained it, and when the object is rebuilt the field simply gets the default value of its type, which is `null` for a reference. That is the behaviour the scenario needs, since the session token must not travel over the network.

The static field is a different case. A static field belongs to the class and not to a particular object, and serialization saves only the state of the object. So the value I see after restoring is whatever the class holds in the JVM that does the restoring. In this program that is the same JVM, which is why the text I assigned a moment earlier shows up. To be sure I was reading this correctly I added `StaticFieldCrossJvmDemo`. One run sets `systemVersion` to `WRITER-JVM-9.9`, serializes the object and saves the bytes to a file. A second, separate run reads that file. Figure 10 shows the second JVM printing `v2.4`, the original value in the class, and a null token. Neither the static value nor the token was in the stream.

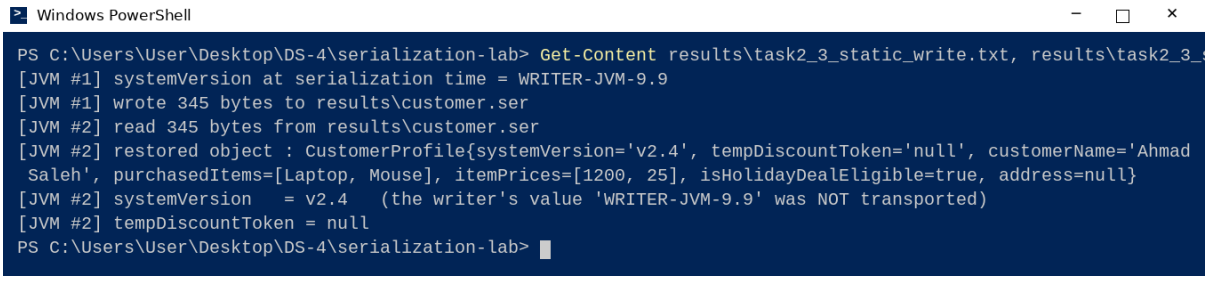


Figure 10. Writing the object in one JVM and reading it in a new one.

Experiment 2: a field whose class does not implement Serializable

I created `Address` without `implements Serializable` and gave `CustomerProfile` an `address` field. With a non-null address, serialization fails with **`java.io.NotSerializableException: Address`** (Figure 9). The exception comes from `ObjectOutputStream.writeObject` while writing, not while reading, because Java serializes the whole graph of objects reachable from the one you pass in and every object on the way has to be serializable. The message names the class that caused the problem, which makes it easy to find. There are a few ways out: make `Address` implement `Serializable`, mark the field `transient` if it does not need to be sent (it then comes back as null), write custom `writeObject` and `readObject` methods, or use `Externalizable`.

Experiment 3: printing the payload as text

Printing `new String(payload)` gives the first block of Figure 11. Most of it is unreadable, but a lot of it is plain text. Because the unprintable bytes make it hard to follow, my program also prints a version where they are replaced by dots, and a hex dump (Figure 12).



Figure 11. The payload printed with new String(payload) and with non-printable bytes replaced by dots.

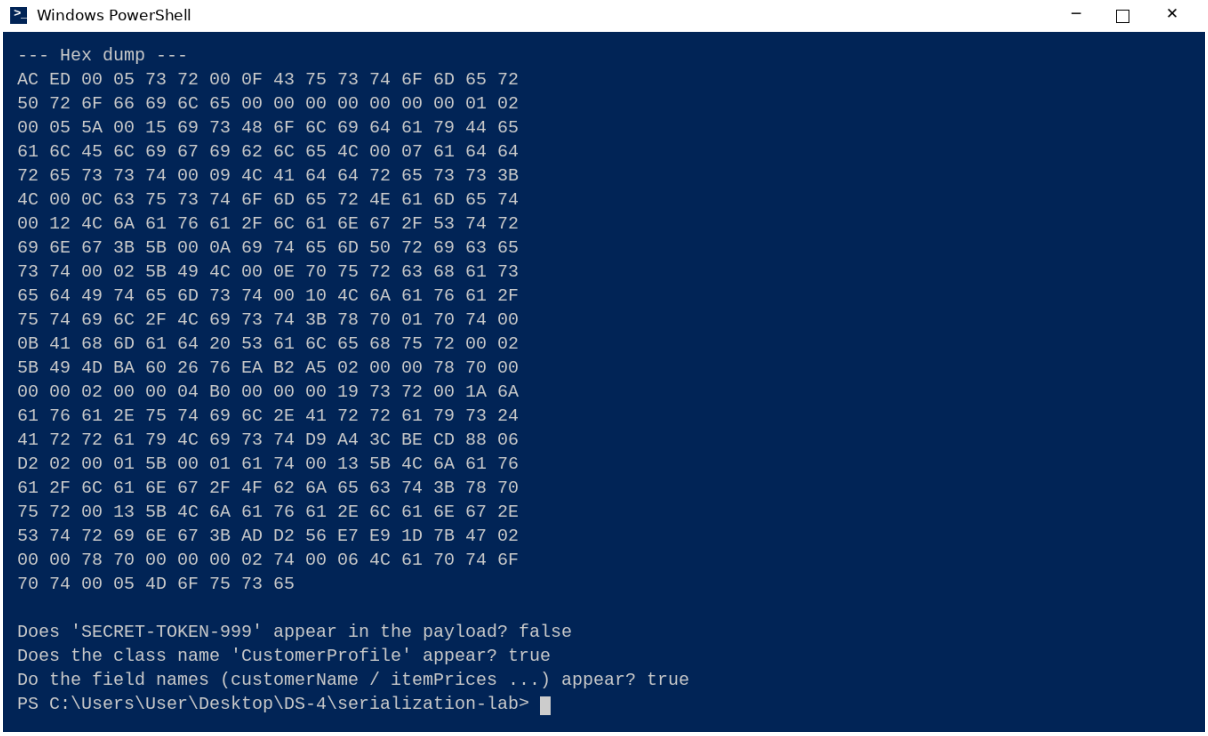


Figure 12. Hex dump of the 345 byte payload and the three checks at the end of Experiment 3.

Reading through it I can pick out the stream header `AC ED 00 05` at the start, the class name `CustomerProfile`, the field names `isHolidayDealEligible`, `address`, `customerName`, `itemPrices` and `purchasedItems`, and their types written out as `Ljava/lang/String;`, `[I` (array of int), `Z` (boolean), `Ljava/util/List;` and `LAddress;`. The internal class `java.util.Arrays$ArrayList`, created by `Arrays.asList`, is there too, and so are the values themselves: `Ahmad` `Saleh`, `Laptop` and `Mouse`. Two things are missing, the token `SECRET-TOKEN-999` and even the name `tempDiscountToken`, because transient and static fields are left out completely. The three checks at the bottom of Figure 12 confirm it: false for the token, true for the class name, true for the field names.

This matters for security. Someone who captures the message learns the class name, its structure and the types of its fields without needing any schema, and the values are readable. Knowing the class names also helps an attacker, because Java deserialization can be abused with crafted streams that make the receiver instantiate classes already on its classpath. A system should never deserialize data from a source it does not trust, or at least restrict the allowed classes with an `ObjectInputFilter`, and the traffic needs TLS because nothing in this format is encrypted.

4. Experiment 3: Google Protocol Buffers

Task 3.1: the schema file

I wrote `customer_profile.proto` as in the lab (listing in the appendix). Every field has its own tag number: `customer_name` is 1 and required, `city` is 2 with the default "Damascus", `purchased_items` is 3 and repeated, `item_prices` is 4 and repeated, and `is_holiday_deal_eligible` is 5 with the default false. As explained in the setup, the Java stub is generated by the Maven plugin when I run `mvn compile`.

The lab uses `protoc` directly, so I also ran it by hand. I took the `protoc` binary that Maven had already downloaded into `target\protoc-plugins`, checked its version (libprotoc 25.5) and generated the stub into a separate folder, so it does not clash with the one from the Maven build: `protoc -I src\main\proto --java_out=generated-manual src\main\proto\customer_profile.proto`. The result is a `CustomerProfileProto.java` of 46,704 bytes, the same size as the file from the Maven build (Figure 13 and Figure 14).

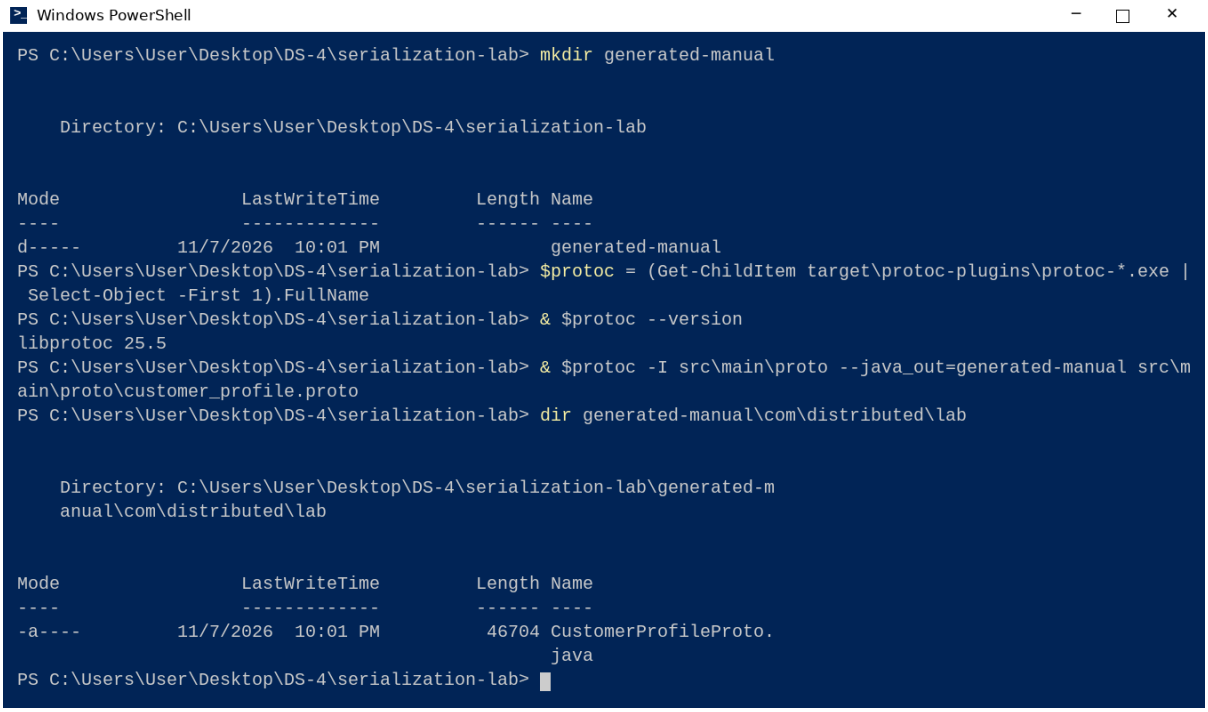


Figure 13. Running protoc by hand with the binary downloaded by Maven.

Task 3.2: the generated class, building the object and measuring

I opened the generated `CustomerProfileProto.java` (46,704 bytes, in `target\generated-sources\protobuf\java\com\distributed\lab\`, next to two folders for the extra schemas of Report question 2) and searched it for the methods I use. Figure 14 shows what the search found. `newBuilder()` is a static method of the message, while `build()` and the setters belong to the nested `Builder` class, for example `setCustomerName` and `setIsHolidayDealEligible`. For the repeated fields there is `addAllPurchasedItems` on the builder and `getPurchasedItemsCount()` on both the message and the builder, and `getItemPrices(int)` reads one price by its index. Optional fields get a `has` method next to the getter. `hasCity()` is the useful one here, because `getCity()` returns the default "Damascus" when the field was never set, so without `hasCity()` there would be no way to tell "not sent" from "sent with the value Damascus". Reading a message is the static `parseFrom(byte[])`. `toByteArray()` is not in this file at all, since it is inherited from the protobuf base class. My search pattern did not return the two list getters, `getPurchasedItemsList()` and `getItemPricesList()`, but `ProtobufLab` calls both and runs, so they exist.

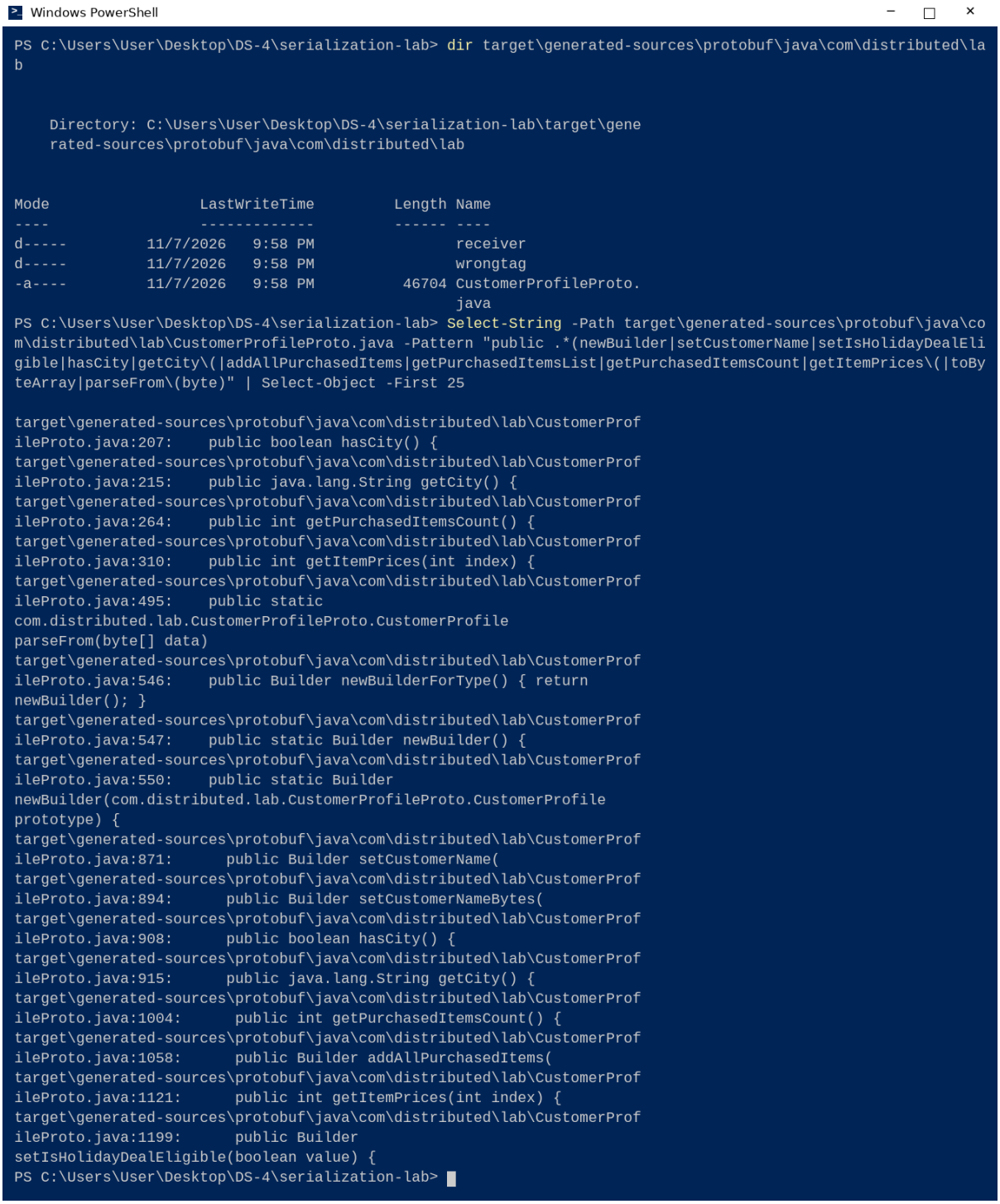


Figure 14. The generated stub folder and the methods found in CustomerProfileProto.java.

`ProtobufLab` builds the message with the builder and writes it out as 35 bytes (Figure 15). After `parseFrom` the name is Ahmad Saleh, and `getCity()` returns Damascus even though I never set it. `hasCity()` is false for the message that was sent, so the city did not take up a single byte on the wire.

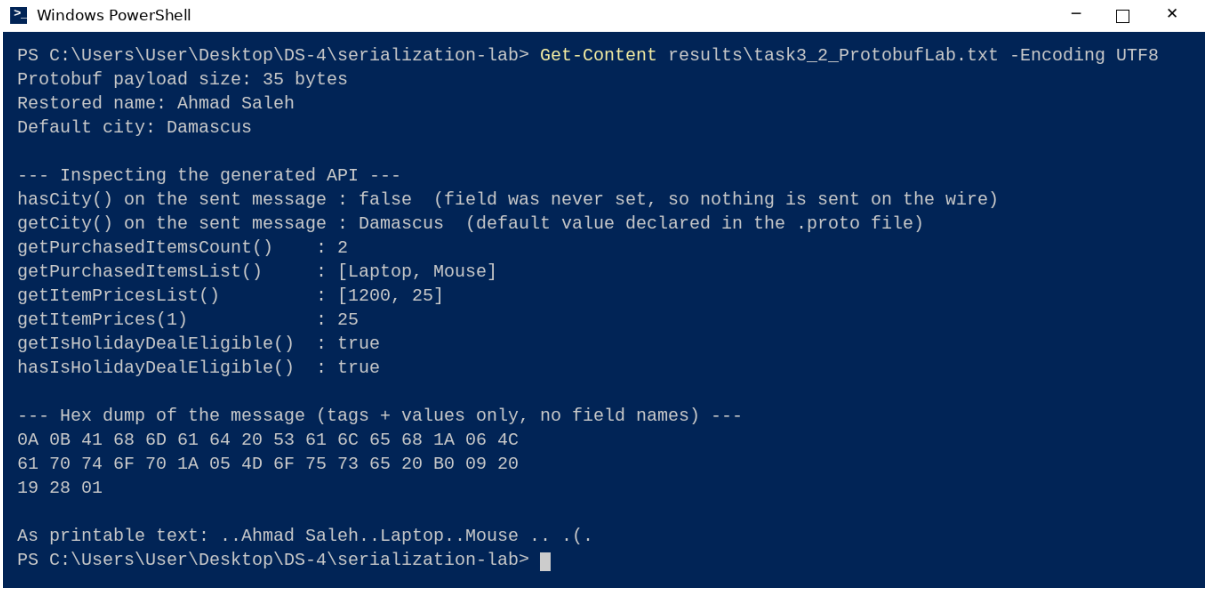


Figure 15. Output of ProtobufLab.

The hex dump at the bottom of Figure 15 is the whole message and it can be read by hand. `0A 0B` is field 1 with a length of 11, followed by the 11 characters of the name. `1A 06` is field 3 with length 6 (Laptop) and `1A 05` is the same field again for Mouse. `20 B0 09` is field 4 as a varint, and `B0 09` decodes to 1200, then `20 19` is the second price, 25. `28 01` is field 5, the boolean true.

Report question 2

(a) Size of the same data in Protobuf, Java Serialization and JSON

`SizeComparison` serializes exactly the same data in the three formats: the name, the two items, the two prices and the flag. The city is left out because the Java class has no city field and Protobuf does not send fields that were never set. The output is in Figure 16.

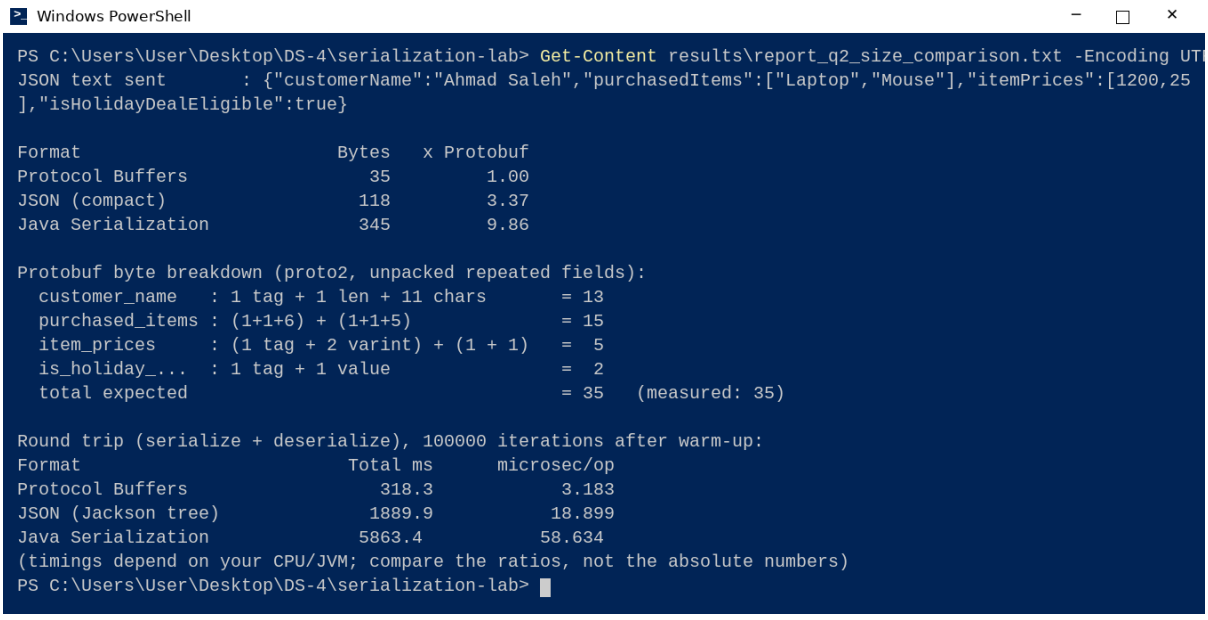


Figure 16. Output of SizeComparison: sizes, the hand calculation for Protobuf and the timing.

Format	Size (bytes)	Size vs Protobuf	Round trip (µs)	Time vs Protobuf
Protocol Buffers	35	1.0x	3.18	1.0x
JSON (Jackson)	118	3.4x	18.90	5.9x
Java Serialization	345	9.9x	58.63	18.4x

The 35 bytes of Protobuf can be checked by hand. The name takes 1 byte for the tag, 1 for the length and 11 characters, so 13. The two items take (1 + 1 + 6) + (1 + 1 + 5) = 15. The two prices take 1 tag byte plus 2 bytes for 1200 and 1 tag byte plus 1 byte for 25, so 5. The flag takes 2. That adds up to 35, and the program measured 35.

Protobuf is the smallest because a field is identified by a one-byte tag instead of its name, numbers are written as varints, fields that are not set are not sent, and no description of the class is included. JSON repeats every field name as text and adds quotes, brackets and commas, which gives 118 bytes. Java serialization is the largest because every message carries the full class descriptor: the class name, the names and types of the fields, the `serialVersionUID` and the descriptors of internal classes such as `Arrays$ArrayList`. For an object this small that overhead is almost the whole message.

The program also times a complete serialize and deserialize round trip, 100,000 times after a warm-up. Protobuf needed about 3.2 microseconds per round trip, JSON with Jackson about 18.9 and Java serialization about 58.6, so Protobuf was roughly six times faster than JSON and eighteen times faster than Java serialization. This is a simple timing loop and not a proper benchmark. In an earlier run I got 3.3, 15.6 and 53.7 microseconds, so the order stays the same but the exact values move, and I only trust the ratios.

(b) Renaming customer_name = 1 to client_full_name = 1 on the receiver only

For this test I made a copy of the `.proto` file with the field renamed to `client_full_name = 1` and generated it as a separate stub, with a different package and class name so that both can live in the same project. The sender is untouched. I took the 35 bytes written by the sender and parsed them with the new receiver class. **Deserialization succeeds** and `getClientFullName()` returns Ahmad Saleh (Figure 17).

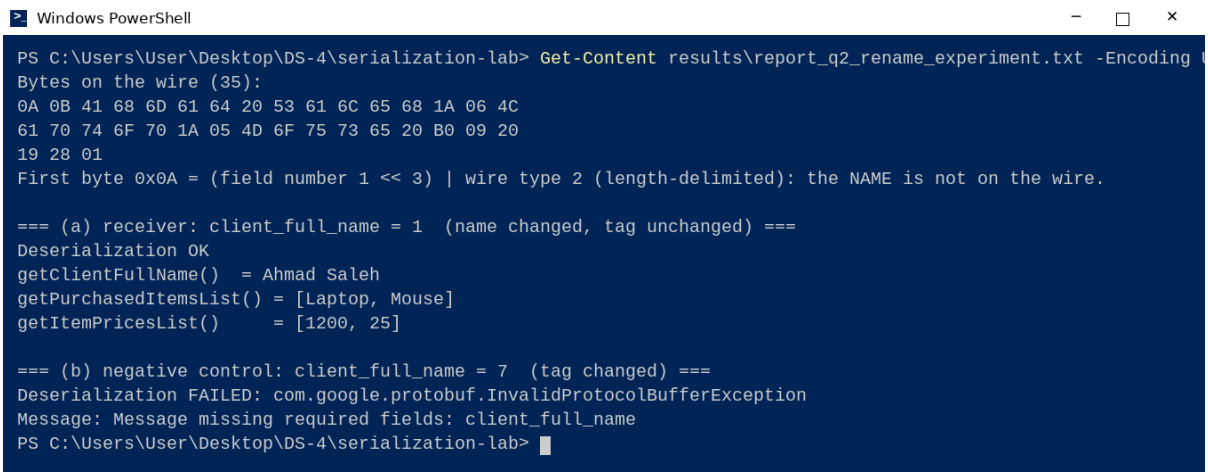


Figure 17. Output of ProtobufRenameExperiment: renamed field (works) and changed tag number (fails).

The reason is that the message contains no field names at all. Each field starts with a key made of the field number and a wire type, `(number << 3) | wire type`. The first byte of the message is `0A`, which is field 1 with wire type 2 (length-delimited), followed by the length 11 and the text. The receiver only matches that number and type against its own schema. The name exists only in the `.proto` file and in the generated code, so renaming a field while keeping its number does not change a single byte on the wire.

As a control I changed the number as well, from 1 to 7, in a third copy of the schema. That one fails with `InvalidProtocolBufferException: Message missing required fields: client_full_name`, because the receiver looks for field 7 and does not find it (the bytes of field 1 are treated as an unknown field). So the contract is the tag number and the type, and the name is only there for the programmer. One caveat is that this applies to the binary format. The JSON and text representations of Protobuf do use field names, so a rename would break those.

5. Evaluation and closing discussion questions

Case 1: a JavaScript web application sends shopping cart data to the backend

JSON is the right choice. The browser reads and writes it natively with `JSON.stringify`, `JSON.parse` and `fetch`, with no library and no build step. A browser cannot use Java serialization at all, and Protobuf would need an extra JavaScript library and generated code in the front end. JSON is also easy to inspect in the developer tools and to test with curl. A shopping cart is small, so the size advantage of Protobuf is not worth the extra tooling. I would still add a JSON Schema or OpenAPI description, validate the input on the server and use HTTPS.

Case 2: a fast prototype with 3 Java nodes on the same JVM version, no external libraries or compilers

Java serialization fits this case best. It is part of the JDK, there is nothing to install and no compiler step, and complex objects can be shared by adding `implements Serializable`. The limits should be known, though. It breaks easily when a class changes (that is what `serialVersionUID` is about), the messages are large (345 bytes against 35 in my test), it ties the system to Java, and deserializing untrusted data is dangerous. For a closed prototype on a trusted network that is acceptable, but I would replace it with Protobuf or JSON before going to production.

Case 3: a large trading system of Java, C++ and Go services needing top security, low bandwidth and a strict schema

Protocol Buffers, ideally together with gRPC. There are official generators for Java, C++ and Go from the same `.proto` file, so the teams share one contract and type mismatches are caught when the code is built rather than in production. The messages are small and fast to parse (35 bytes against 118 for JSON in my test, and about six times faster than Jackson). The compatibility rules let tools such as `buf breaking` detect changes that would break other teams. On security, Protobuf is not encryption, so the protection has to come from TLS or mutual TLS and from authentication and authorization on the transport, which gRPC supports. What Protobuf does add is that it only fills in fields of a known message type and does not rebuild arbitrary Java objects, which removes most of

the deserialization attack surface that Java serialization has. Size limits and validation of the values should still be applied.

Case 4: why must the tag number of a deleted optional field never be reused in Protocol Buffers?

Because the number is the only identity a field has on the wire. There is no name and no version to tell an old field from a new one. Suppose field `X = 6` is deleted and later the number 6 is given to a new field `Y`. Old data in databases, Kafka topics or files, and senders that have not been updated, still write number 6 meaning `X`, and the new receiver reads it as `Y`. If the wire types differ, for example a string and an integer, parsing fails or the field is dropped. If the type is the same, for example both `int32`, the result is silent corruption: a `discount_percent` of 15 is read as `loyalty_points` of 15 and nothing complains. It also works the other way round, with a new sender and an old receiver. No check at run time can catch it, because the number and the type both match.

The protection is to write `reserved 6;` (and `reserved "old_name";`) in the `.proto` file when a field is removed, so the compiler refuses any later use of that number or name. New fields get new numbers. The numbers 1 to 15 take only one byte, so they are valuable, but even those must not be reused.

Appendix A: source listings

CustomerProfile.java

```
import java.io.*;
import java.util.Arrays;
import java.util.List;

/** Task 2.1 - the serializable class (exactly as in the lab + the Address field of task 2.3).
*/
public class CustomerProfile implements Serializable {
    private static final long serialVersionUID = 1L;

    // Class-level (static) field - NOT serialized
    public static String systemVersion = "v2.4";

    // Temporary (transient) field - NOT serialized
    private transient String tempDiscountToken;

    // Core instance fields - serialized automatically
    private String customerName;
    private List<String> purchasedItems;
    private int[] itemPrices;
    private boolean isHolidayDealEligible;

    // Task 2.3 (experiment 2): Address does NOT implement Serializable.
    // It stays null in every normal run, so tasks 2.2 / 3.x are not affected;
    // the experiment sets it to a real object to trigger the exception.
    private Address address;

    // No-arg constructor
    public CustomerProfile() {}

    public CustomerProfile(String customerName, List<String> purchasedItems,
                           int[] itemPrices, boolean isHolidayDealEligible,
                           String token) {
        this.customerName = customerName;
```

```
        this.purchasedItems = purchasedItems;
        this.itemPrices = itemPrices;
        this.isHolidayDealEligible = isHolidayDealEligible;
        this.tempDiscountToken = token;
    }

    public String getCustomerName() { return customerName; }

    public String getTempDiscountToken() { return tempDiscountToken; }

    public Address getAddress() { return address; }

    public void setAddress(Address address) { this.address = address; }

    @Override
    public String toString() {
        return "CustomerProfile{" +
                "systemVersion='" + systemVersion + '\'' +
                ", tempDiscountToken='" + tempDiscountToken + '\'' +
                ", customerName='" + customerName + '\'' +
                ", purchasedItems=" + purchasedItems +
                ", itemPrices=" + Arrays.toString(itemPrices) +
                ", isHolidayDealEligible=" + isHolidayDealEligible +
                ", address=" + address +
                '}';
    }
}
```

Address.java

```
/**
 * Task 2.3 (experiment 2): a class that deliberately does NOT implement java.io.Serializable.
 * Using it as a non-null field of CustomerProfile makes ObjectOutputStream fail with
 * java.io.NotSerializableException: Address
 */
public class Address {
    private String street;
    private String city;

    public Address() {}

    public Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    @Override
    public String toString() {
        return "Address{street='" + street + "', city='" + city + "'}";
    }
}
```

customer_profile.proto

```
syntax = "proto2";

package distributed.lab;

option java_package = "com.distributed.lab";
option java_outer_classname = "CustomerProfileProto";

message CustomerProfile {
    required string customer_name = 1;
    optional string city = 2 [default = "Damascus"];
    repeated string purchased_items = 3;
    repeated int32 item_prices = 4;
    optional bool is_holiday_deal_eligible = 5 [default = false];
```

```
}
```

customer_profile_receiver.proto (field renamed, tag unchanged)

```
syntax = "proto2";

// Receiver-side copy of the schema: only the FIELD NAME changed, the tag number is still 1.
// Different package / outer class name only so both stubs can live in one Maven project.
package distributed.lab.receiver;

option java_package = "com.distributed.lab.receiver";
option java_outer_classname = "CustomerProfileReceiverProto";

message CustomerProfile {
    required string client_full_name = 1;
    optional string city = 2 [default = "Damascus"];
    repeated string purchased_items = 3;
    repeated int32 item_prices = 4;
    optional bool is_holiday_deal_eligible = 5 [default = false];
}
```

StrictPriceReceiver.java (the receiver of Task 1.2)

```
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

/**
 * The "receiver" microservice of task 1.2.
 * It expects  {"prices": [1200.00, 25.00, 85.00]}  - an array of JSON NUMBERS.
 * Strings such as "1200" are rejected (no silent coercion), which is what exposes the missing
schema.
 */
public final class StrictPriceReceiver {

    private static final ObjectMapper MAPPER = new ObjectMapper();

    private StrictPriceReceiver() {}

    public static double[] parsePrices(String json) throws DeserializationException {
        JsonNode root;
        try {
            root = MAPPER.readTree(json);
        } catch (JsonProcessingException e) {
            throw new DeserializationException("Malformed JSON: " + e.getOriginalMessage(),
e);
        }

        if (root == null || !root.isObject()) {
            throw new DeserializationException("Expected a JSON object at $ but found "
                    + (root == null ? "nothing" : root.getNodeType()));
        }
        JsonNode prices = root.get("prices");
        if (prices == null || !prices.isArray()) {
            throw new DeserializationException("Expected an ARRAY at $.prices but found "
                    + (prices == null ? "nothing (field missing)" : prices.getNodeType()));
        }

        double[] result = new double[prices.size()];
        for (int i = 0; i < prices.size(); i++) {
            JsonNode element = prices.get(i);
            if (!element.isNumber()) {
                throw new DeserializationException(String.format(
                        "Cannot deserialize $.prices[%d]: expected NUMBER but found %s
(value=%s)",
                        i, element.getNodeType(), element));
            }
            result[i] = element.asDouble();
```

```
        }
        return result;
    }

    public static double product(double[] values) {
        double p = 1.0;
        for (double v : values) {
            p *= v;
        }
        return p;
    }
}
```

Appendix B: how to reproduce

From the project folder in PowerShell: build once, run all experiments, then start the server in one window and send the request from another.

```
mvn compile
.\run-all.bat
.\run.bat AnalyzeServer
curl.exe -X POST http://localhost:8080/api/analyze -H "Content-Type: application/json" -d
"@customer_request.json"
```